Chap 12. 프로세스 동기화

12-1. 동기화,평행 프로세서
12-2. 동기화 기법
12-3. 세마포어, 상호배제

학습목표

- 병행성의 원리를 이해한다.
- 병행 프로세스 수행과 관련하여 상호배제를 해결하는 방법을 알아본다.

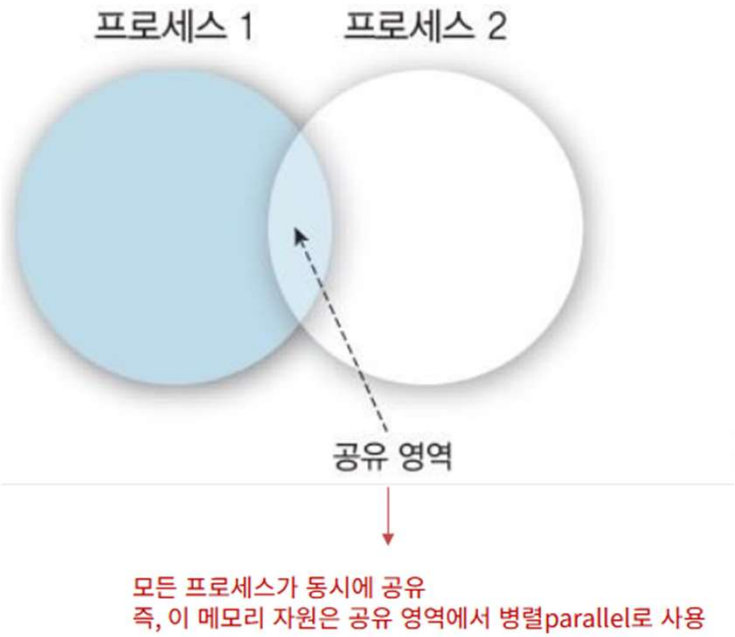
병행 프로세스와 동기적 프로세스

- 병행 프로세스
  - 여러 프로세스가 동시에 실행되는 것.
  - 순서나 타이밍이 보장되지 않으며, 서로 독립적으로 진행
  - 비동기적으로 동작

병행 프로세스와 동기적 프로세스

■ 병행 프로세스

- 운영체제가 프로세서를 빠르게 전환, 프로세서 시간 나눠 마치 프로세스 여러 개를 동시에 실행하는 것처럼 보이게 하는 것
  - 여러 프로세스가 동시에 실행되는 것.
  - 순서나 타이밍이 보장되지 않으며, 서로 독립적으로 진행
  - 비동기적으로 동작
  - 메모리 공유 영역



병행 프로세스와 동기적 프로세스

- 병행 프로세스
  - 여러 프로세스가 동시에 실행되는 것.
  - 순서나 타이밍이 보장되지 않으며, 서로 독립적으로 진행
  - 비동기적으로 동작

- 동기적 프로세스
  - 프로세스들이 순서와 타이밍을 맞춰 실행되는 것
  - 한 작업이 끝나야 다음작업이 시작되거나
  - 특정 시점에 서로 보조를 맞추는 방식

프로세스 동기화

- **여러 프로세스가 공유자원에 동시에 접근할때 발생하는 문제를 해결하고 올바르게 제어하는 메커니즘**
- **상호배제(공유 자원 보호)와 실행 순서 제어가 목적임**

- **핵심 개념 : 경쟁 조건**
  - 두 개 이상의 프로세스가 공유 데이터에 동시에 접근하면 실행순서에 따라 결과가 달라지는 문제 발생

- **동기화가 사용되는 목적**
  - 상호배제(공유 자원 보호)와 실행 순서 제어가 목적임
  - 병행 프로세스이므로 동기화가 필요한 것

병행 프로세스의 개념

▪ 병행 프로세스 종류

- 독립 프로세스
  - 단일 처리 시스템에서 수행하는 병행 프로세스, 다른 프로세스에 영향 주고받지 않으면서 독립 실행
  - 다른 프로세스, 데이터와 상태 공유 않고 동작도 재현 가능
  - 주어진 초기값에 따라 항상 동일한 결과
  - 중지 후 변동 없이 다시 시작 가능
  - 독립 실행할 수 있는 프로세스
    - 단일 프로그래밍 : 프로세서를 사용 중이던 프로세스 완료 후 다른 프로세스 실행
    - 다중 프로그래밍 : 프로세스 여러 개가 프로세서 하나 공유. 공유하지 않는 상태일 때 디스패치 순서 상관 없음
    - 다중 처리 : 프로세서 2개 이상 사용하여 동시에 프로그램 여러 개를 병렬 실행.
    - 프로세스는 한 번에 프로세서 하나에서 실행하지만, 동일한 시스템에서는 서로 다른 시간에 서로 다른 프로세서에서 실행 가능

병행 프로세스의 개념

■ 병행 프로세스 종류

- 협력 프로세스
  - 다른 프로세스와 상호작용하며 특정 기능 수행하는 비동기적 프로세스
  - 제한된 컴퓨터 자원의 효율성 증대, 계산 속도 향상, 모듈적 구성 강화, 개별 사용자의 여러 작업 동시에 수행 편의성 제공에 사용
  - 간단한 예 : 두 프로세스의 동일한 파일 사용 시
  - 프로세스 하나가 파일에서 읽기 수행 동안 다른 프로세스가 해당 파일에 쓰기하면 서로 영향
  - 병행 프로세스들이 입출력장치, 메모리, 프로세서, 클록 등 자원을 서로 사용 시 충돌 발생
  - 충돌을 피하기 위한 프로세스의 상호작용 형태
  - ❶ 프로세스는 서로 인식하지 못하는 경쟁 관계 유지. 다중 프로그래밍 환경이 대표적인 예로, 운영체제가 자원 경쟁 고려하여 동일한 디스크나 프린터로 접근 조절
  - ❷ 프로세스는 입출력 버스를 비롯한 개체를 공유하는 단계에서 간접적으로 서로 관계 인식. 이때 다른 프로세스에서 얻은 정보에 의존, 프로세스의 타이밍에 영향. 프로세스들은 개체 공유에 따른 협력 필요
  - ❸ 프로세스에는 서로 인식하고 프로세스끼리 통신할 수 있는 기본 함수 있음. 프로세스가 서로 협력 관계에 있으면 직접 통신 가능, 병행해서 함께 동작 가능

병행 프로세스의 해결과제(수정)

■ 병행성

- 여러 프로세스를 이용하여 작업을 수행하는 것
- 시스템 신뢰도 높이고 처리 속도 개선, 처리 능력 높이는 데 중요

■ 병행 프로세스의 문제

- 공유 자원 상호 배타적 사용(프린터, 통신망 등은 한순간에 프로세스 하나만 사용)
- 병행 프로세스 간의 협력이나 동기화(상호배제도 동기화의 한 형태)
- 두 프로세스 간 데이터 교환을 위한 통신
- 동시에 수행하는 다른 프로세스의 실행 속도와 관계 없이 항상 일정한 실행 결과 보장
- (결정성determinacy)확보
- 교착 상태 해결, 병행 프로세스들의 병렬 처리 능력 극대화
- 실행 검증 문제 해결
- 병행 프로세스 수행 과정에서 발생하는 상호배제 보장

실행순서 제어를 위한 동기화

- **개념**
  - 프로세스 A의 작업결과를 프로세스 B가 사용해야 하는 경우
  - 인위적인 순서 강제 제어 필요
  - 세마포어를 이용한 순서 제어

- **세마포어란?**
  - 정수 변수 하나로 표현
  - P 연산 : 대기. 정수 값을 1감소.프로세스가 자원을 사용하려 할 때 호출
  - V연산 : 프로세스 깨움. 정수값을 1 증가. 프로세스가 자원 사용을 마쳤을 때 호출출
  - 지금 몇 개의 프로세스가 더 들어올 수 있는지를 나타내는 숫자
  - 숫자의 증감으로 프로세스의 접근과 대기를 제어함

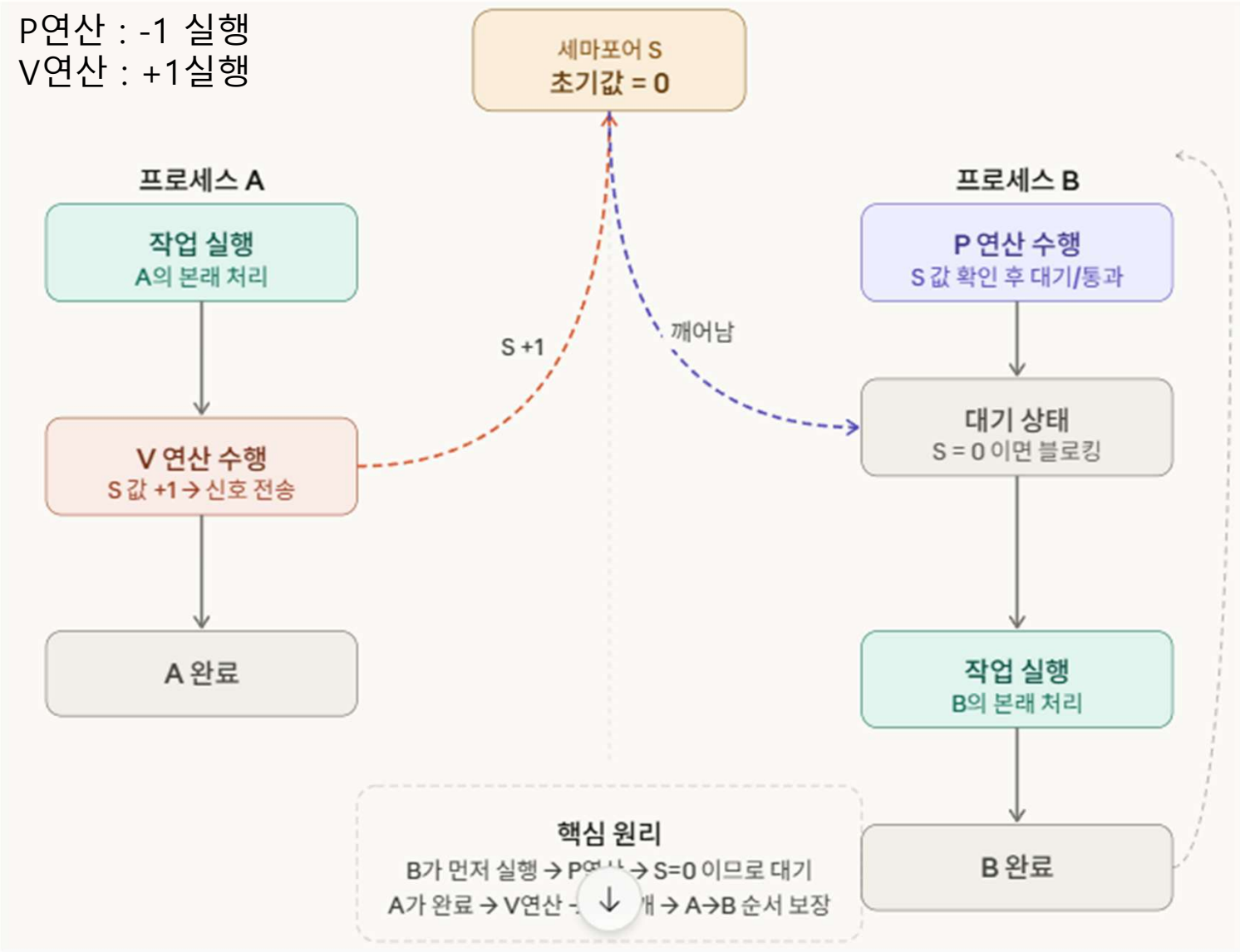
- **세마포어의 두 종류**
  - 이진 세마포어 :한 번에 한 프로세스만 접근 가능
  - 카운팅 세마포어 : 최대 N개의 프로세스 동시 접근 가능

세마포어를 이용한 순서 제어

- **개념**
  - 세마포어 변수를 0으로 초기화
  - 프로세스 B가 먼저 실행되면 P연산 수행
    - 세마포어가 음수가 되어 대기 상태로 들어감
  - 이후 A가 실행을 마치고 V연산 수행
    - 세마포어가 증가하고 대기 중이던 B가 깨어나 실행

  - A가 먼저 실행되면 V연산으로 세마포어가 1이되고
  - B는 P연산 후 바로 통과하여 대기 없이 실행
  - 어느쪽이 먼저 실행되더라도 결과적으로 A-->B 순서가 보장됨

프로세스 실행과 세마포어 변수 변화

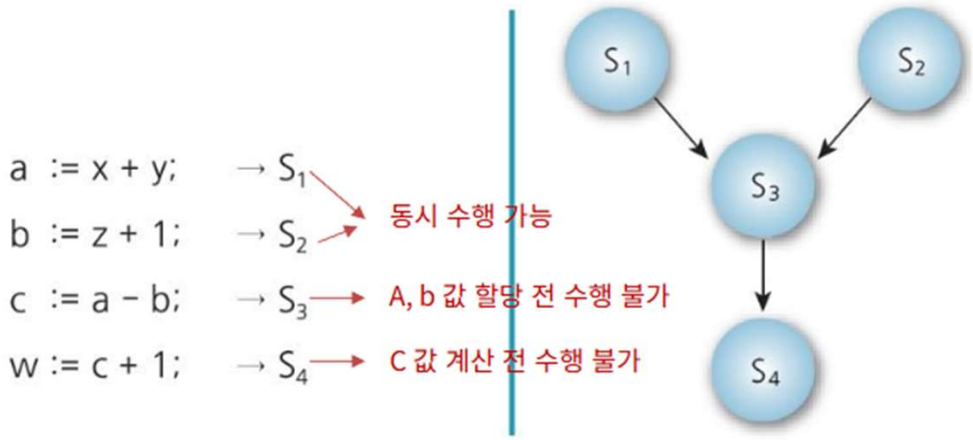


병행 프로그램을 설명할 때 선형그래프로 설명하는 이유

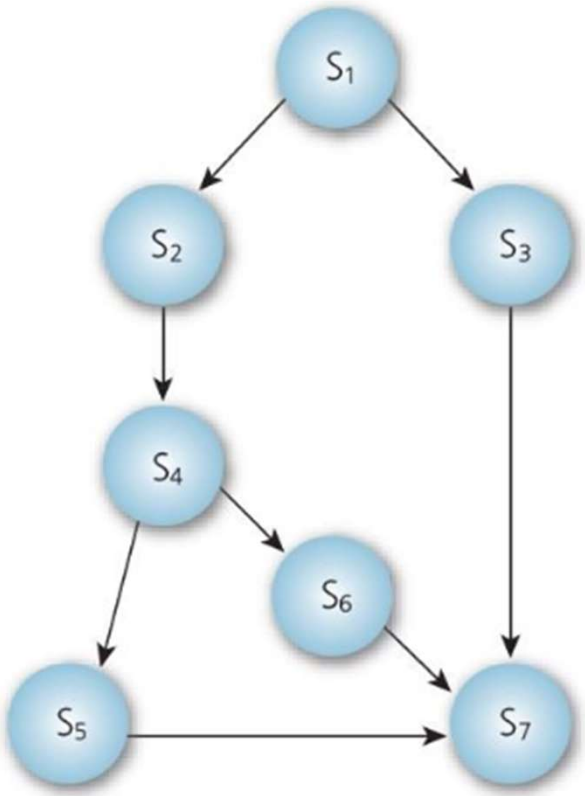
- 순차 프로그램은 코드 흐름을 따른다.
- 병행 프로그램은 여러 프로세스가 동시에 실행되므로 순서 예측이 어려움
- 선형 그래프(프로세스 그래프) 사용 이유
- 가능한 실행 경로의 전체 공간을 볼 수 있음
- 임계구역의 위치를 직사각형 영역으로 표시 가능
- 안전한 실행 경로와 위험한 경로 구분 가능

선형 그래프와 병행 프로그램

- **선행그래프**precedence graph
  - 선행 제약의 논리적 표현
  - 프로세스 : 프로세스 집합과 이것의 선행 제약precedence constraint 두 가지 요소로 정의
  - 선행 제약 : 프로세스를 순서대로 다른 상태로 옮기는 것
    - 두 프로세스에 선행 제약이 없으면 이 둘은 독립적이므로 병행 실행 가능
  - 순차적 활동을 표현하는 방향성 비순환 그래프
  - 선행 그래프에서 노드는 소프트웨어 작업이거나 동시에 실행할 수 있는 프로그램 명령



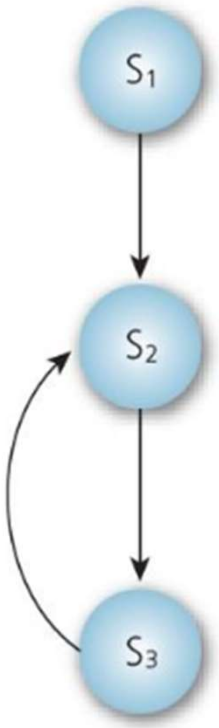
선형그래프와 병행 프로그램



(a) 선행 그래프

- S_2와 S_3은 S_1이 끝난 후에 수행한다.
- S_4는 S_2가 끝난 후에 수행한다.
- S_5와 S_6은 S_4가 끝난 후에 수행한다.
- S_7은 S_5, S_6, S_3이 끝난 후에 수행하고, S_3은 S_2, S_4, S_5, S_6과 병행하여 수행할 수 있다.

(b) 선행 관계



순차실행

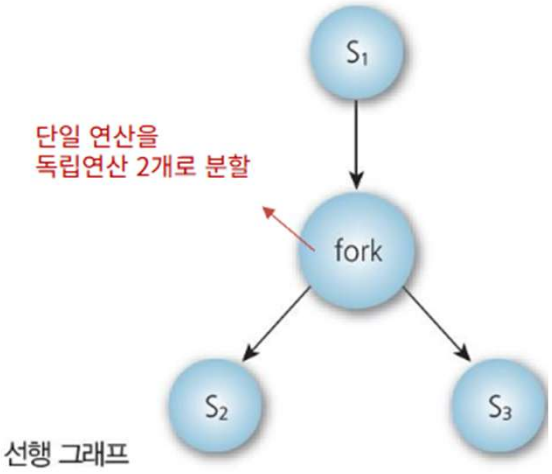
▪ fork와 join 구조

- 선행 그래프는 2차원이라 프로그램에는 사용 곤란
- 선행관계란 "A가 끝나야 B가 시작될 수 있다."는 실행순서의 의존성
- 선행 관계 명시 위해 fork와 join 구조, 병행 문장 (parbegin/parend) 등 다른 방법 필요
- 콘웨이Conway(1963년)와 데니스Dennis (1966년), 혼Van Horn(1966년)이 소개
- fork와 join : 프로그램의 선행관계를 실제 코드에서 구현하는 수단
- fork : 실행 흐름을 두 갈래로 나누는 명령
- Join :흩어진 실행 흐름을 하나로 합하는 명령

fork 명령어

```
S1;
fork L;
S2;
:
L : S3;
```

(a) 알고리즘



(b) 선행 그래프

join 명령어

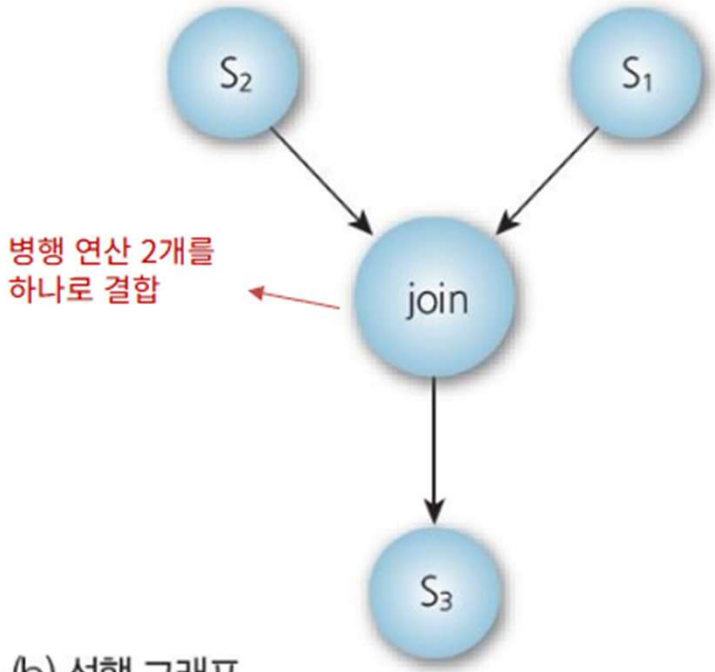
```
...
count := count - 1;
if count = 0 then quit;
```

0 이 아닌 정수값

종료 명령어

```
        count := 2;
        fork L1;
        :
        S1;
        goto L2;
L1 :    S2;
L2 :    join count;
        S3;
```

(a) 알고리즘

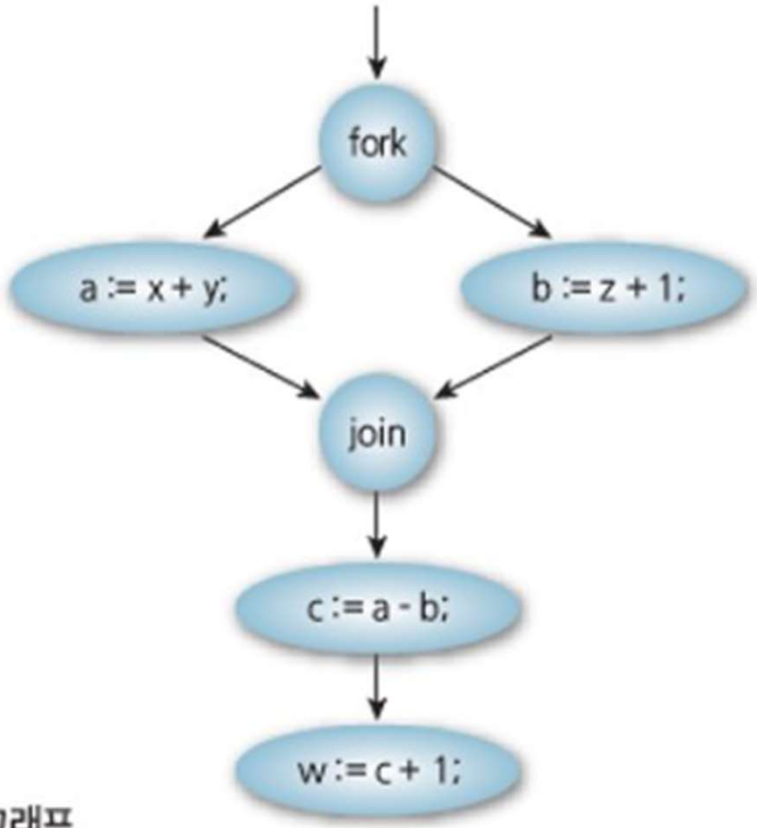


(b) 선행 그래프

선형그래프와 병행 프로그램

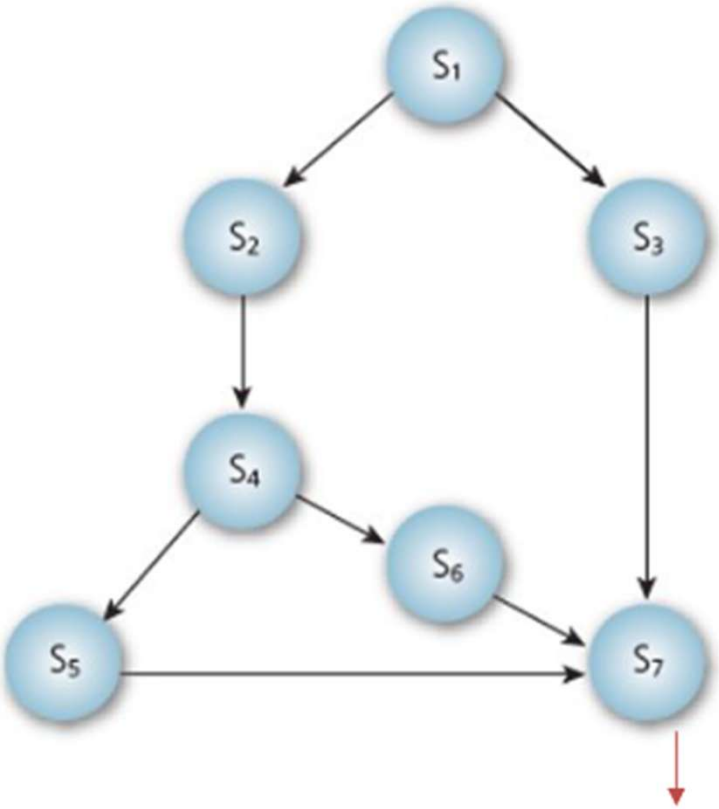
```
        count := 2;
        fork L1;
        a := x + y;
        goto L2;
L1 :    b := z + 1;
L2 :    join count;
        c := a - b;
        w := c + 1;
```

(a) 알고리즘



(b) 선행 그래프

선형그래프와 병행 프로그램



(a) 선행 그래프

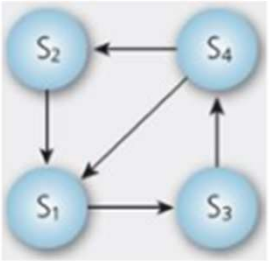
유일한 join 노드

```
      S1;
      count := 3;
      fork L1;
      S2;
      S4;
      fork L2;
      S5;
      goto L3;
L2 :  S6;
      goto L3;
L1 :  S3;
L3 :  join count;
      S7;
```

(b) 알고리즘

유입 정도와 유출 정도

유입 정도in-degree는 방향 그래프에서 들어오는 간선 수이고, 유출 정도out-degree는 방향 그래프에서 나가는 간선 . 예를 들어, 다음 그래프에서 S1의 유입 정도 는 2, 유출 정도는 1이다.



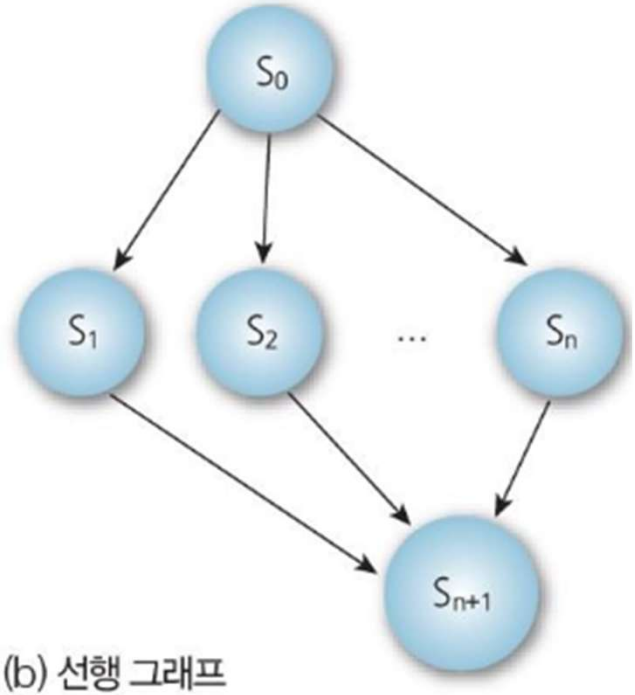
■ 병행 문장

- 하나의 프로세스가 여러 병렬 프로세스로 퍼졌다가 다시 하나로 뭉쳐지는 것을 나타냄
- 대표적인 예 : 다익스트라(1965년)가 제안한 parbegin/ parend
- 일반적인 형태

```
parbegin S1; S2; . . . . . . ; Sn; parend;
```

S_0; parbegin S_1; S_2; . . . . . ; S_n; parend; S_{n+1};

a) 일반 구조의 병행 문장



(b) 선행 그래프

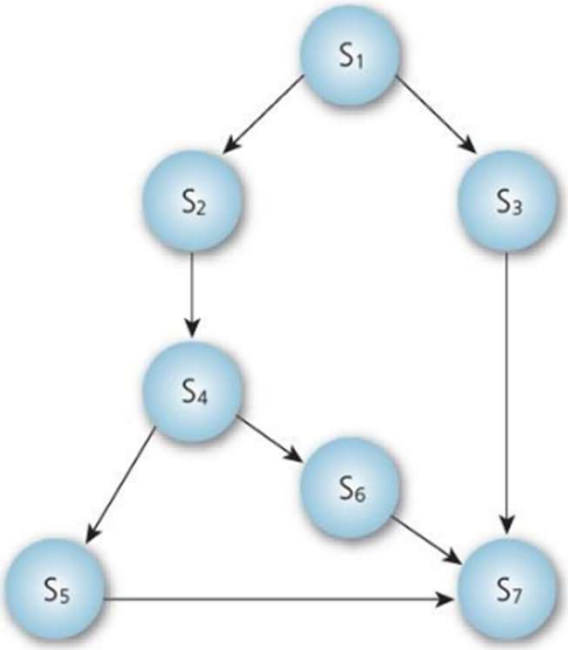
선형그래프와 병행 프로그램

```
a := x + y;
b := z + 1;
c := a - b;
w := c + 1;
```

(a) 간단한 산술 알고리즘

```
parbegin
    a := x + y;
    b := z + 1;
parend;
    c := a - b;
    w := c + 1;
```

(b) parbegin/parend 구조 알고리즘



(a) 선행 그래프

```
S1;
parbegin
    S3;
    begin
        S2;
        S4;
        parbegin
        S5;
        S6;
        parend;
    end;
parend;
S7;
```

(b) 알고리즘

뮤텍스 락(Mutex Lock)

- 상호배제(Mutual Exclusion)의 줄임말
- 여러 스레드가 공유자원에 동시에 접근하는 것을 막기 위한 동기화 메커니즘
- 경쟁조건이 발생 시 한 번에 하나의 스레드만 임계구역에 진입하도록 보장

동작 원리

```
스레드 A: [락 획득] → [임계 구역 실행] → [락 해제]
스레드 B:            [대기 중...]       → [락 획득] → [실행]
```

1. 스레드가 임계 구역 진입 전 **lock()** 호출
2. 이미 잠겨 있으면 **블로킹(대기)**
3. 작업 완료 후 **unlock()** 호출
4. 대기 중인 스레드가 락 획득

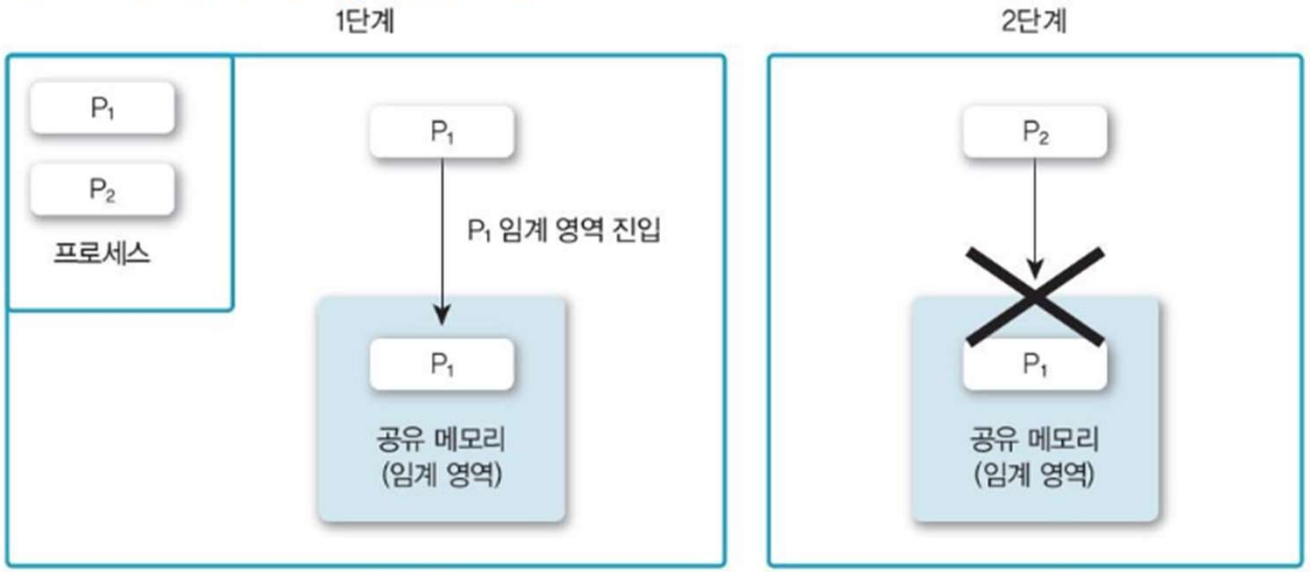
cpp

```cpp
#include <mutex>
#include <thread>

std::mutex mtx;
int counter = 0;

void increment() {
    mtx.lock();          // 락 획득
    counter++;           // 임계 구역
    mtx.unlock();        // 락 해제
}

// 더 안전한 방법: RAII 패턴
void safe_increment() {
    std::lock_guard<std::mutex> lock(mtx); // 자동 해제
    counter++;
}
```

■ 상호배제mutual exclusion의 개념

- 병행 프로세스에서 프로세스 하나가 공유 자원 사용 시 다른 프로세스들이 동일한 일을 할 수 없도록 하는 방법
- 읽기 연산은 공유 데이터에 동시에 접근해도 문제 발생 않음.
- 동기화 : 변수나 파일은 프로세스별로 하나씩 차례로 읽거나 쓰도록 해야 하는데, 공유 자원을 동시에 사용하지 못하게 실행을 제어하는 방법.
  - 동기화는 순차적으로 재사용 가능한 자원을 공유하려고 상호작용하는 프로세스 사이에서 나타남
  - 동기화로 상호배제 보장할 수 있지만, 이 과정에서 교착 상태와 기아 상태가 발생할 수 있음

상호배제의 구체적인 예



임계자원critical resource : 두 프로세스가 동시에 사용할 수 없는 공유 자원
임계영역critical section : 임계 자원에 접근하고 실행하는 프로그램 코드 부분

상호배제의 조건

두 프로세스는 동시에 공유 자원에 진입 불가.

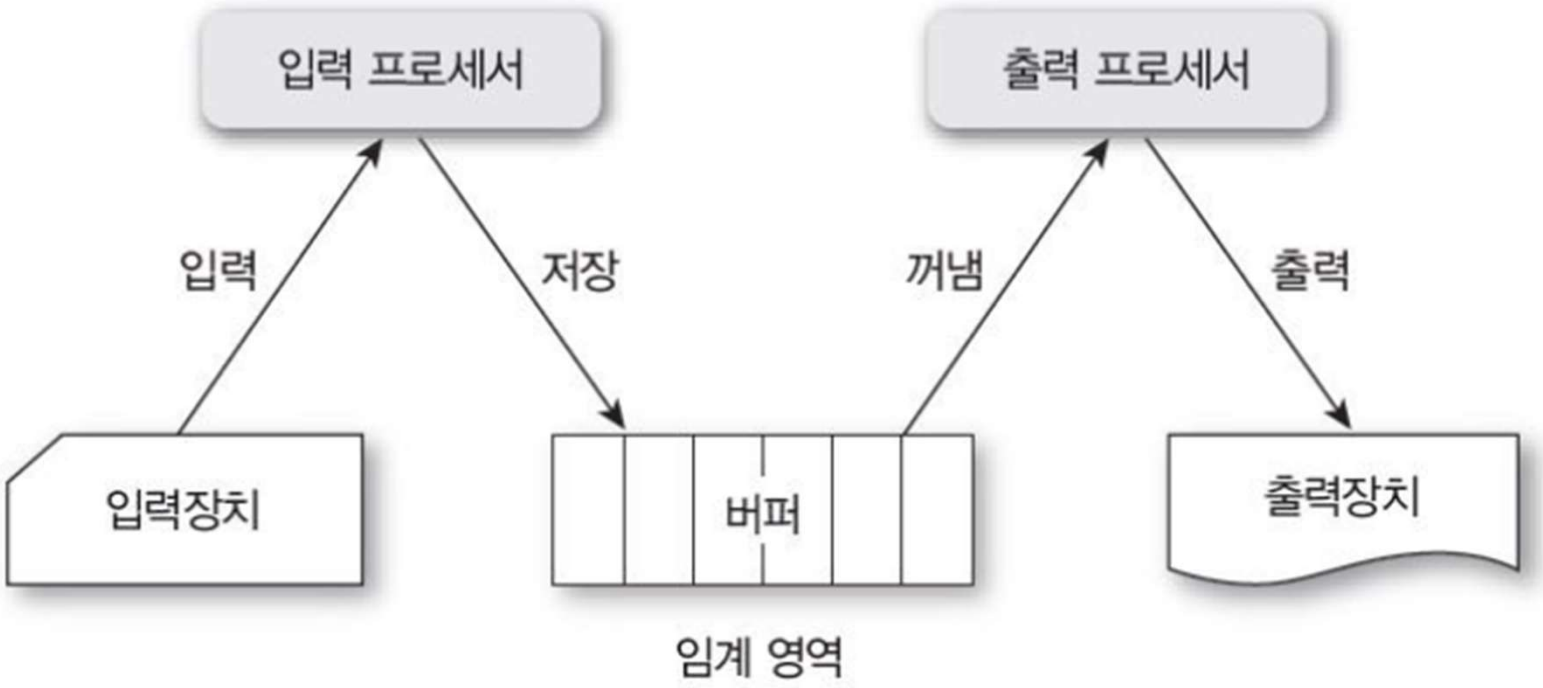
프로세스의 속도나 프로세서 수에 영향 받지 않음

공유 자원을 사용하는 프로세스만 다른 프로세스 차단 가능

프로세스가 공유 자원을 사용하려고 너무 오래 기다려서는 안 됨

▪ 임계 영역의 개념

- 다수의 프로세스 접근 가능하지만, 어느 한 순간에는 프로세스 하나만 사용 가능
- 예



임계구역

- 야러 스레드가 동시에 접근하면 아되는 코드 영역
- 주로 공유자원(변수, 파일, DB등)을 읽거나 쓰는 부분이 해당됨

```
counter = 0 (초기값)

스레드 A: counter 읽기 (0)
스레드 B: counter 읽기 (0)   ← A가 아직 쓰기 전에 읽음!
스레드 A: counter + 1 = 1, 저장
스레드 B: counter + 1 = 1, 저장

결과: counter = 1  ← 2가 되어야 하는데 잘못됨!
```

```cpp
// 진입 구역: 락 획득 시도
mtx.lock();

// ────────────────────────────
//         임계 구역
counter++;    // 공유 자원 접근
// ────────────────────────────

// 퇴출 구역: 락 해제
mtx.unlock();

// 나머지 구역: 공유 자원과 무관한 코드 (동시 실행 가능)
do_other_work();
```

임계구역이 만족해야 할 3가지 조건

1.상호배제

- 한 스레드가 임계구역 실행 중이면 다른 스레드는 절대 진입 불가

2.진행

- 임계구역에 아무도 없으면 진입하려는 스레드는 즉시 들어갈 수 있어야 함
- 불필요한 대기 금기

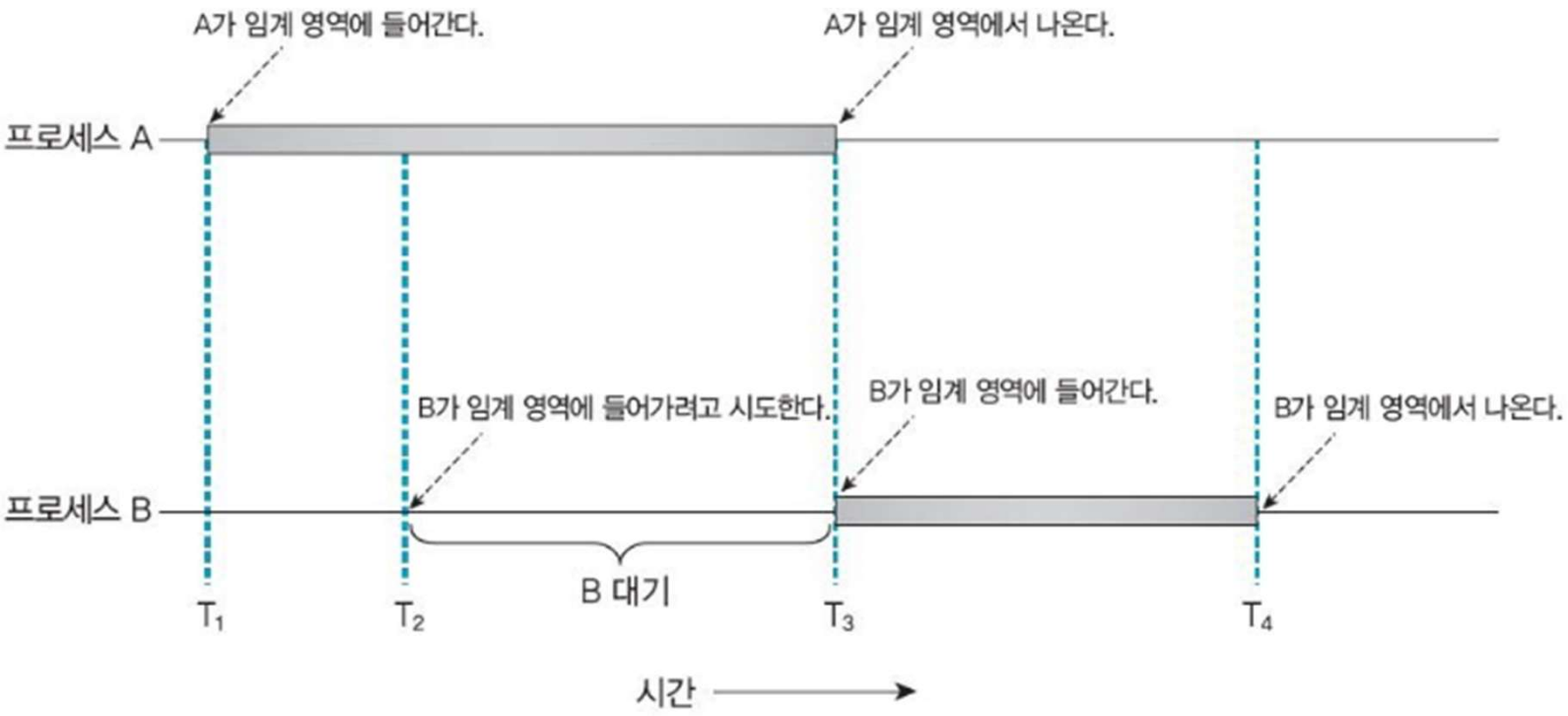
3.한정 대기

- 대기 중인 스레드는언젠가는 반드시 진입할 수 있어야 함
- 무한 대기 금지

4.임계구역 범위는 최소화해야 함

▪ 임계 영역 이용한 상호배제

- 간편하게 상호배제 구현 가능(자물쇠와 열쇠 관계)
  - 프로세스가 진입하지 못하는 임계 영역(자물쇠로 잠근 상태)
- 어떤 프로세스가 열쇠 사용할 수 있는지 확인하려고 검사 하는 동작과 다른 프로세스 사용 금지하는 동작으로 분류
- 예



▪ 병행 프로세스에서 영역 구분

```
do {
        while (turn != i);      ← 진입 영역
        /* 임계 영역 */
        turn = j;               ← 탈출 영역
        /* 나머지 영역 */
} while (TRUE)
```

▪ 임계 영역의 조건

1. 상호배제 : 어떤 프로세스가 임계 영역에서 작업 중, 다른 프로세스 임계 영역 진입 불가
2. 진행 : 임계 영역에 프로세스가 없는 상태에서 어떤 프로세스가 들어갈지 결정
3. 한정 대기 : 다른 프로세스가 임계 영역을 무한정 기다리는 상황 방지 위해 임계 영역에 한 번 들어갔던 프로세스는 다음에 임계 영역에 다시 들어갈 때 제한

생산자-소비자 문제

- 생산자는 데이터를 만들어 버퍼에 넣고 소비자는 버퍼에서 데이터를 꺼내 처리하는 고전적인 동기화문제
- 기본구조
- [생산자] ---> [버퍼(공유자원)] -->[소비자]

  데이터 생성 한정된 크기 데이터 처리

발생하는 문제들

1.버퍼가 가득 찼을 때 : 소비자가 꺼낼 때까지 대기

2.버퍼가 비었을 때 : 생상자가 넣을 때까지 대기해야 함

3.동시 접근 문제 : Race Condition 발생

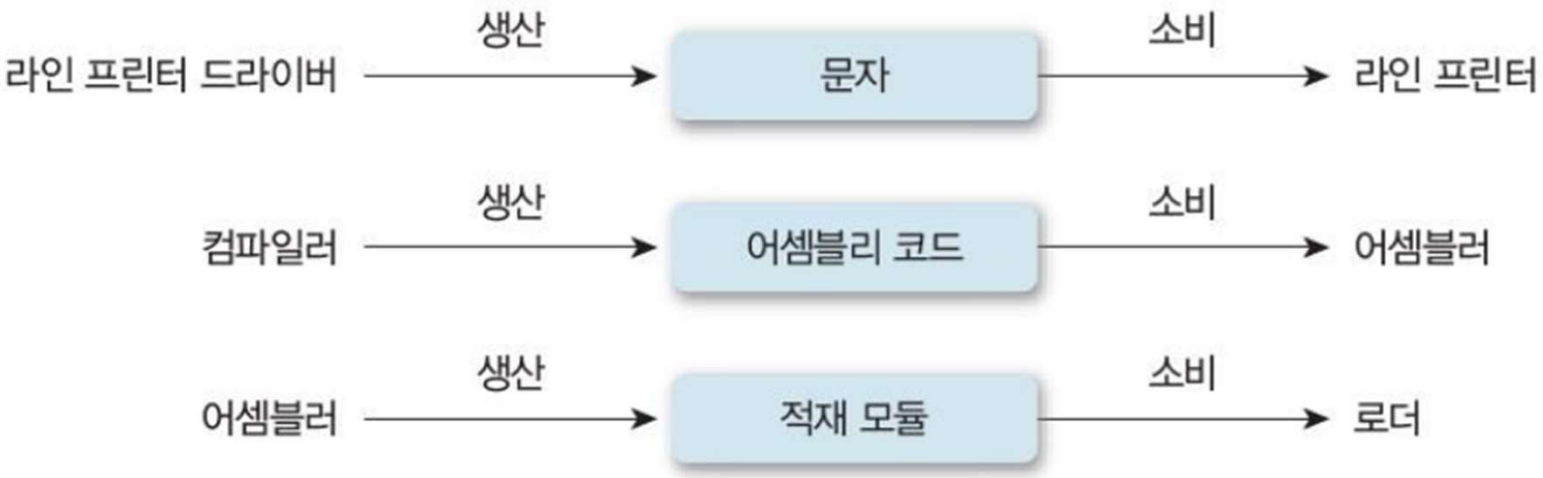
: 조건 변수를 사용하여 해결(뮤텍스만 사용할 경우 한계 발생)

실생활 예시

생산자	버퍼	소비자
웹 서버 요청 수신	요청 큐	워커 스레드
키보드 입력	입력 버퍼	OS 처리
카메라 프레임 캡처	프레임 버퍼	영상 처리
로그 생성	로그 큐	파일 기록

▪ 생산자·소비자 문제

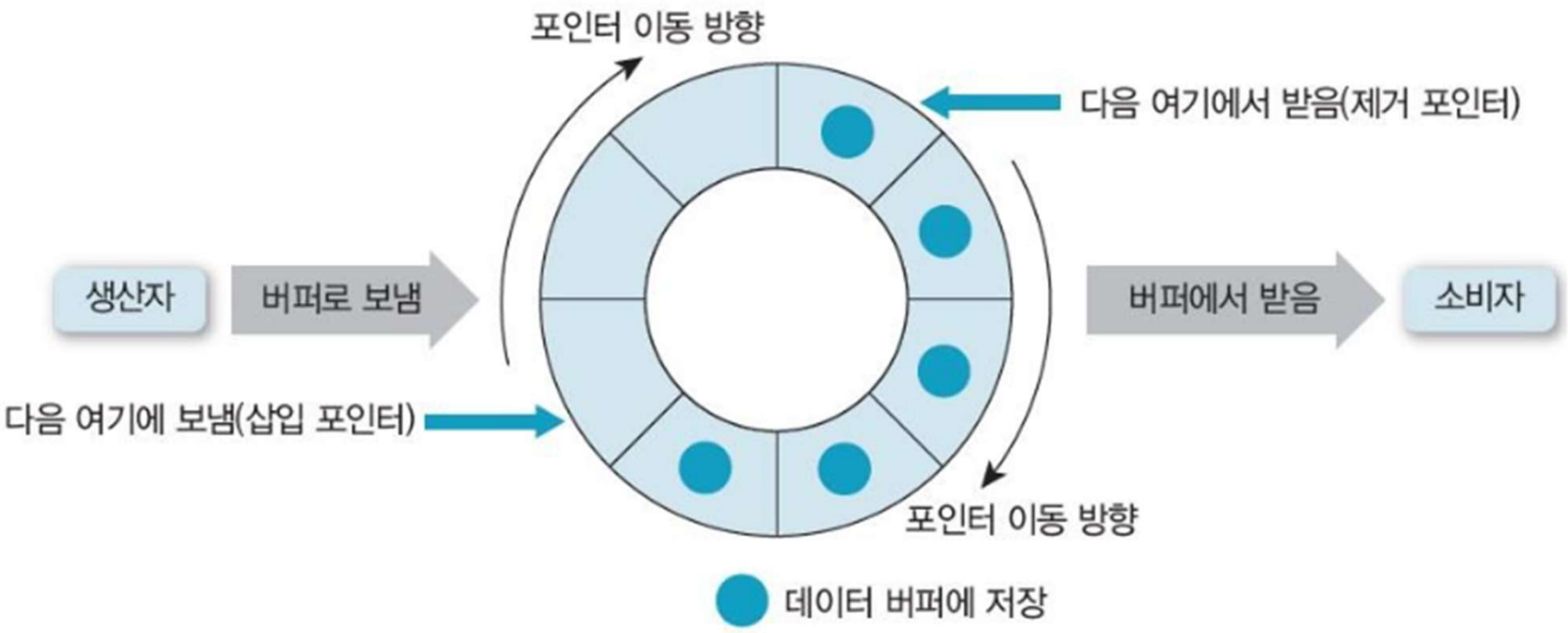
- 운영체제에서 비동기적으로 수행하는 모델
- 생산자 프로세스가 생산한 정보를 소비자 프로세스가 소비하는 형태
- 예 :



생산자 소비자 프로세스의 관계

▪ 생산자의 소비자에게 데이터 전송

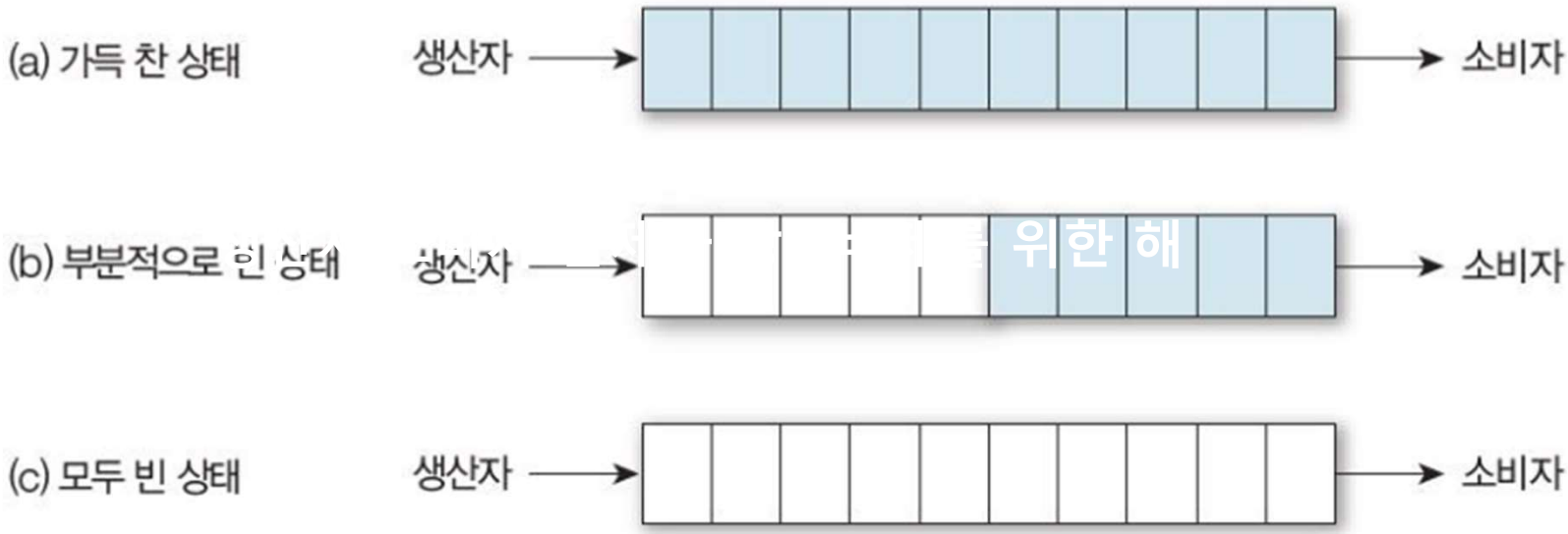
- 소비자가 데이터를 받을 준비를 마칠 때까지 생산자는 버퍼로 데이터 전송



공유 버퍼를 이용한 데이터의 전송과 수신

▪ 생산자와 소비자의 공유 버퍼

- 생산자는 버퍼가 꽉 차면 더 이상 생산 불가, 소비자는 버퍼가 비면 데이터 소비 불가
- 공유 버퍼의 상태

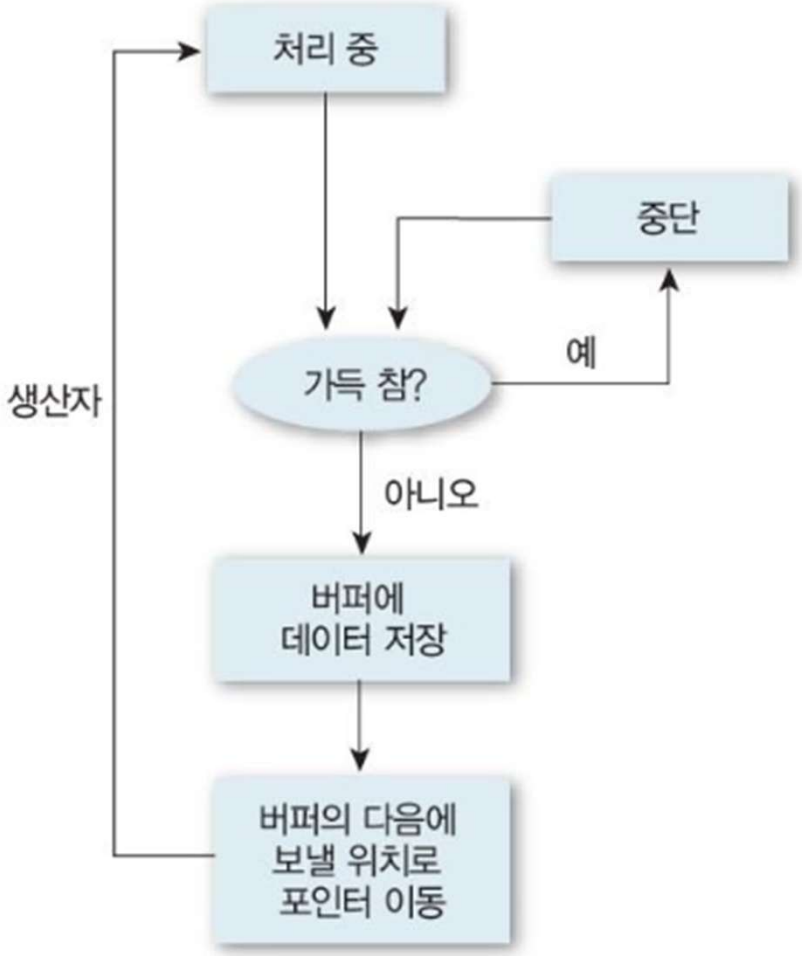


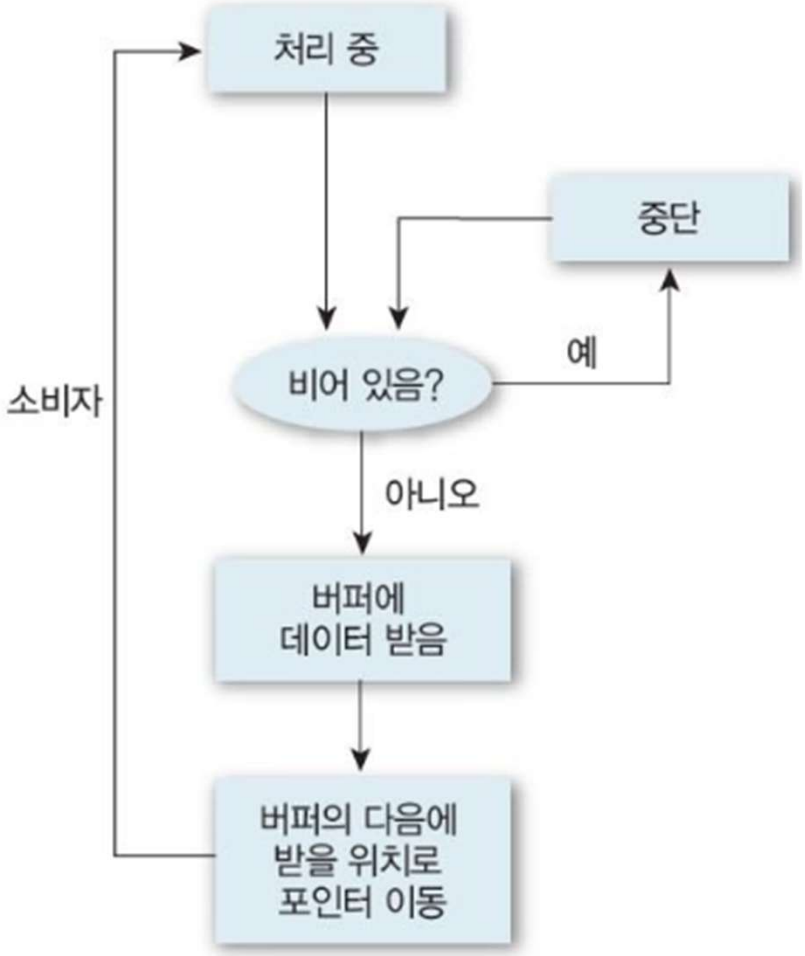
공유버퍼의 3가지 상태

생산자.소비자 문제와 상호배제를 위한 해결시도(초기)

무한 버퍼

- 생산자와 소비자가 독립적으로 알고리즘 수행





생산자.소비자 문제와 상호배제를 위한 해결시도(초기)

공유데이터 버퍼선언

```
#define BUFFER_SIZE 10    // 버퍼 크기
typedef struct {
    DATA data;
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int counter = 0;
```

생산자 프로세스

```
item nextProduced;

while (true) {
    // 버퍼가 가득 차 아무 일도 하지 않음
    while (counter == BUFFER_SIZE);
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

소비자 프로세스

```
item nextConsumed;

while (true) {
    // 버퍼가 비어 아무 일도 하지 않음
    while (counter == 0);
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
}
```

■ 경쟁 상태race condition

▪ 경쟁상태의 개념

- 여러 프로세스가 동시에 공유 데이터에 접근 시, 접근 순서에 따라 실행 결과 달라지는 상황 말함.
- 공유데이터에 마지막으로 남는 데이터의 결과 보장할 수 없는 상황
- 장치나 시스템이 둘 이상의 연산 동시 실행 시, 어느 프로세스를 마지막으로 수행한 후 결과를 저장했느냐에 따라 오류가 발생하므로 적절한 순서에 따라 수행 해야 함
- 읽기와 쓰기 명령을 거의 동시에 실행해야 한다면, 기본적으로 읽기 명령을 먼저 수행 후 쓰기 명령 수행하는 접근

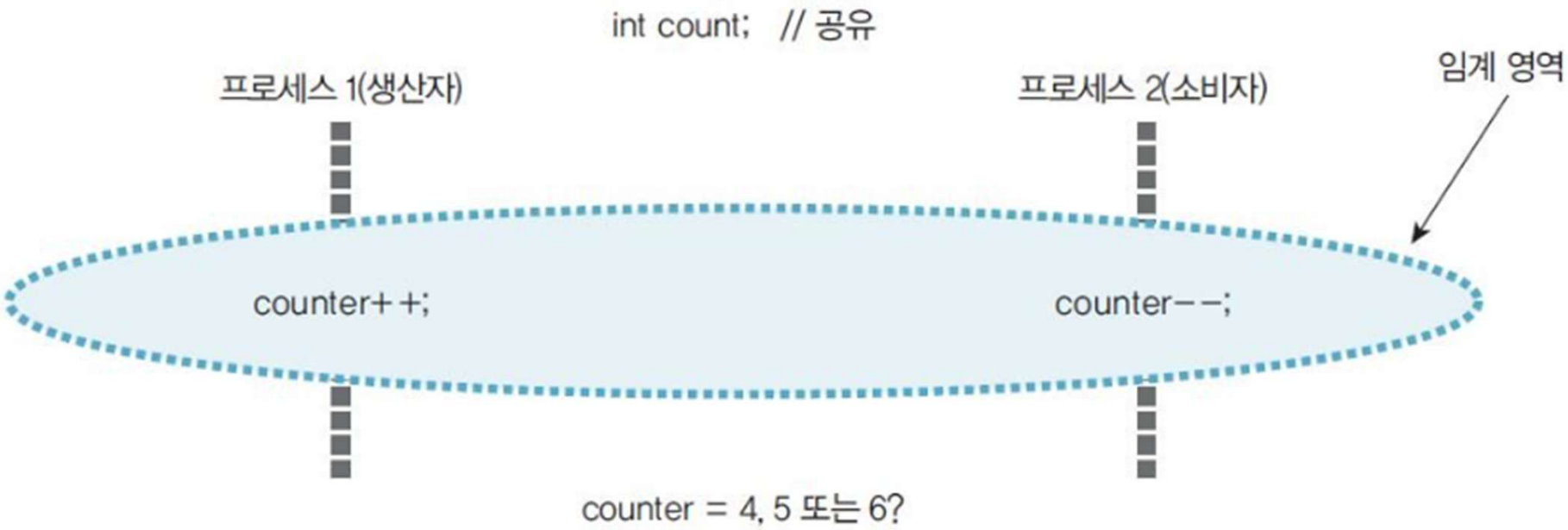
▪ 경쟁 상태의 예방

- 병행 프로세스들을 동기화해야 함(임계 영역 이용한 상호배제로 구현)
- 즉, 공유 변수 counter를 한 순간에 프로세스 하나만 조작할 수 있도록 해야하는 임계 영역과 counter 연산하는 부분

  을 임계 영역으로 설정하여 상호배제하는 방법으로 해결

생산자.소비자 문제와 상호배제를 위한 해결시도(초기)

- 생산자·소비자 문제에서 임계 영역



상호배제 방법

수준	방법	종류
고급	소프트웨어로 해결	• 데커의 알고리즘 • 램포트의 베이커리(빵집) 알고리즘 • 크누스의 알고리즘 • 핸슨의 알고리즘 • 다익스트라의 알고리즘
	소프트웨어가 제공 : 프로그래밍 언어와 운영체제 수준에서 제공	• 세마포 • 모니터
저급	하드웨어로 해결 : 저급 수준의 원자 연산	TestAndSet[TAS](테스)

■ 데커의 알고리즘 개념

- 두 프로세스가 서로 통신하려고 공유 메모리를 사용하여 충돌 없이 단일 자원을 공유할 수 있도록 허용하는 것
- 다익스트라(한 출발점에서 다른 모든 노드까지의 최단 경로를 찾는 알고리즘)로 임계 영역 문제에 적용
- 병행 프로그래밍 상호배제 문제의 첫 번째 해결책
- 각 프로세스 플래그 설정 가능, 다른 프로세스 확인 후 플래그 재설정 가능
- 프로세스가 임계 영역에 진입하고 싶으면 플래그 설정하고 대기

■ 데커의 알고리즘 특징

- 뮤텍스나 OS의 도움없이, 순수하게 소프트웨어 로직으로만 두 스레드의 상호배제를 보장하는 최초의 알고리즘
- 특별한 하드웨어 명령문 필요 없음
- 임계 영역 바깥에서 수행 중인 프로세스가 다른 프로세스들이 임계 영역 진입 막지 않음
- 임계 영역에 들어가기를 원하는 프로세서 무한정 기다리게 하지 않음

데커 알고라즘

핵심 아이디어

두 가지 변수를 조합해서 상호 배제를 구현합니다.

```
flag[2]  → 각 스레드가 "나 임계구역 들어가고 싶어" 의사 표현
turn     → 충돌 시 누가 먼저 들어갈지 결정
```

코드

cpp

```cpp
bool flag[2] = {false, false};  // 진입 희망 표시
int turn = 0;                   // 우선권

void process_0() {
    while (true) {

        flag[0] = true;          // "나 들어가고 싶어"

        while (flag[1]) {        // 상대방도 원하면
            if (turn != 0) {     // 내 차례가 아니면
                flag[0] = false; // 양보
                while (turn != 0) {} // 내 차례 될 때까지 대기
                flag[0] = true;  // 다시 의사 표현
            }
        }

        // ─── 임계 구역 ───
        critical_section();
        // ──────────────────

        turn = 1;                // 다음 차례는 상대방
        flag[0] = false;         // 임계구역 나옴
    }
}

void process_1() {
    while (true) {

        flag[1] = true;          // "나 들어가고 싶어"

        while (flag[0]) {        // 상대방도 원하면
            if (turn != 1) {     // 내 차례가 아니면
                flag[1] = false; // 양보
                while (turn != 1) {} // 내 차례 될 때까지 대기
                flag[1] = true;  // 다시 의사 표현
            }
        }

        // ─── 임계 구역 ───
        critical_section();
        // ──────────────────

        turn = 0;                // 다음 차례는 상대방
        flag[1] = false;         // 임계구역 나옴
    }
}
```

단계별 동작 흐름

단계별 동작 흐름

Case 1 — 한 스레드만 진입 원할 때

```
P0: flag[0] = true
    flag[1] == false → 아무도 없음
    → 바로 임계구역 진입 ✅
```

Case 2 — 동시에 진입 원할 때

```
P0: flag[0] = true
P1: flag[1] = true

P0: flag[1] == true → 충돌 감지!
      turn == 1 → 내 차례 아님 → 양보
      flag[0] = false, 대기

P1: flag[0] == false → 상대 양보함
    → 임계구역 진입 ✅

P1: turn = 0, flag[1] = false
P0: turn == 0 → 내 차례!
    flag[0] = true → 임계구역 진입 ✅
```

임계구역 3가지 조건 만족 여부

1. 상호 배제 ✅

flag와 turn 조합으로
두 프로세스가 동시에 임계구역 진입 불가

2. 진행 ✅

임계구역이 비어있으면
flag가 false → 즉시 진입 가능

3. 한정 대기 ✅

turn 변수가 번갈아 가며 바뀌므로
무한정 대기하는 스레드 없음

한계점

한계	설명
2개 스레드만 가능	3개 이상이면 알고리즘 구조 자체가 복잡해짐
바쁜 대기(Busy-wait)	대기 중 CPU를 계속 소모
현대 CPU에서 불안정	컴파일러/CPU의 명령어 재정렬로 오동작 가능

데커의 알고리즘을 이용한 상호 배제

```
// 프로세스가 공유하는 데이터 flag[] : 부울(boolean) 배열, turn : 정수
flag[0] = false;
flag[1] = false;
turn = 0;                    // 공유 변수, 0 또는 1

// 프로세스 P0:              // 프로세스 P0의 임계 영역 진입 절차
❶ flag[0] = true;            // P0의 임계 영역 진입 표시
❷ while (flag[1] == true) {  // P1의 임계 영역 진입 여부 확인
      if (turn == 1) {       // P1이 진입할 차례가 되면
       ❹ flag[0] = false;    // 플래그를 재설정하여 P1에 진입 순서 양보
       ❺ while (turn == 1) { // turn을 바꿀 때까지 대기
             // 바쁜 대기
         }
         flag[0] = true;     // P1이 임계 영역에 재진입 시도
      }
}
❸ /* 임계 영역 */;
turn = 1;                    // P1에 진입 순서 제공
flag[0] = false;             // P0의 임계 영역 사용 완료 지정
/* 나머지 영역 */;           // P0이 나머지 영역 수행
```

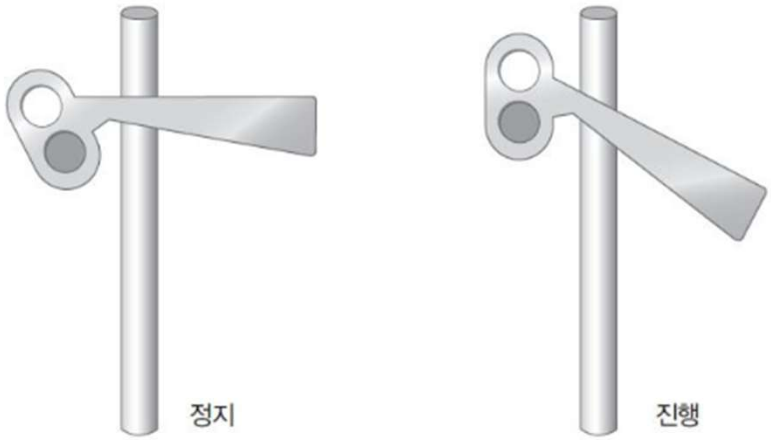
❶에서 P0은 flag[0]을 true로 설정, 자신이 임계 영역으로 들어간다는 사실 알림
❷에서 while 문 검사하여 P1의 임계 영역 진입 여부 확인. P1의 flag[1]이 false이면
❸ P0이 임계 영역으로 진입하고, true이면
❹ P1이 임계 영역에 진입할 차례라서 플래그 false로 재설정 후
❺ while 문에서 순환하며 대기
여기서 공유 변수 turn은 두 프로세스 P0과 P1이 동시에 임계 영역으로 들어가려고 충돌하는 것 방지

■ 데커의 알고리즘을 사용한 상호배제2

```
// 프로세스 P1
flag[1] = true;
while (flag[0] == true) {
    if (turn == 0) {
        flag[1] = false;
        while (turn == 0) {
            // 바쁜 대기
        }
        flag[1] = true;
    }
}
/* 임계 영역 */;
turn = 0;
flag[1] = false;
/* 나머지 영역 */;
```

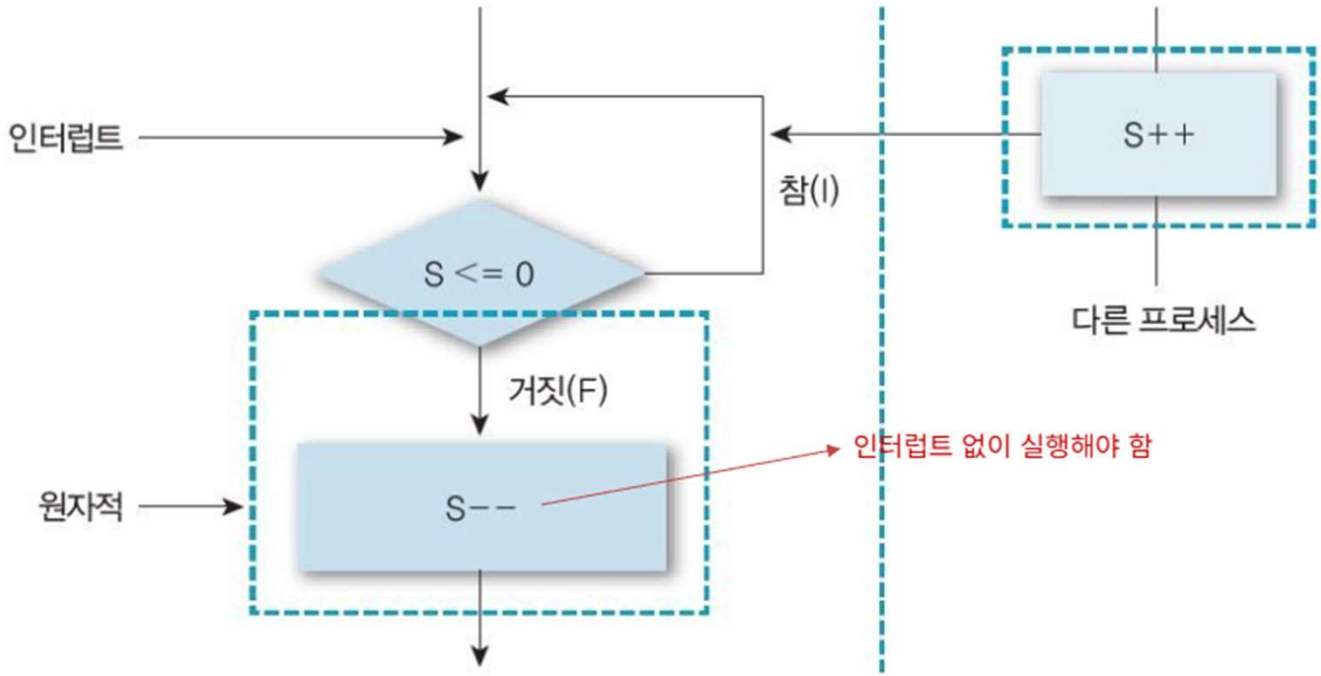
▪ 세마포 개념과 동작

- 다익스트라가 테스 명령어의 문제 해결을 위해 제안
- 상호배제 및 다양한 연산의 순서도 제공
- 세마포 값은 true나 false로, P와 V연산과 관련. 네덜란드어로 P는 검사Proberen, V 증가Verhogen 의미. 음이 아닌 정수 플래그 변수
- 세마포를 의미하는 S는 표준 단위 연산 P(프로세스 대기하게 하는 wait 동작, 임계 영역에 진입하는 연산)와 V(대기 중인 프로세스 깨우려고 신호 보내는 signal 동작, 임계 영역에서 나오는 연산)로만 접근하는 정수 변수
- 예 : 차단기와 같은 역할



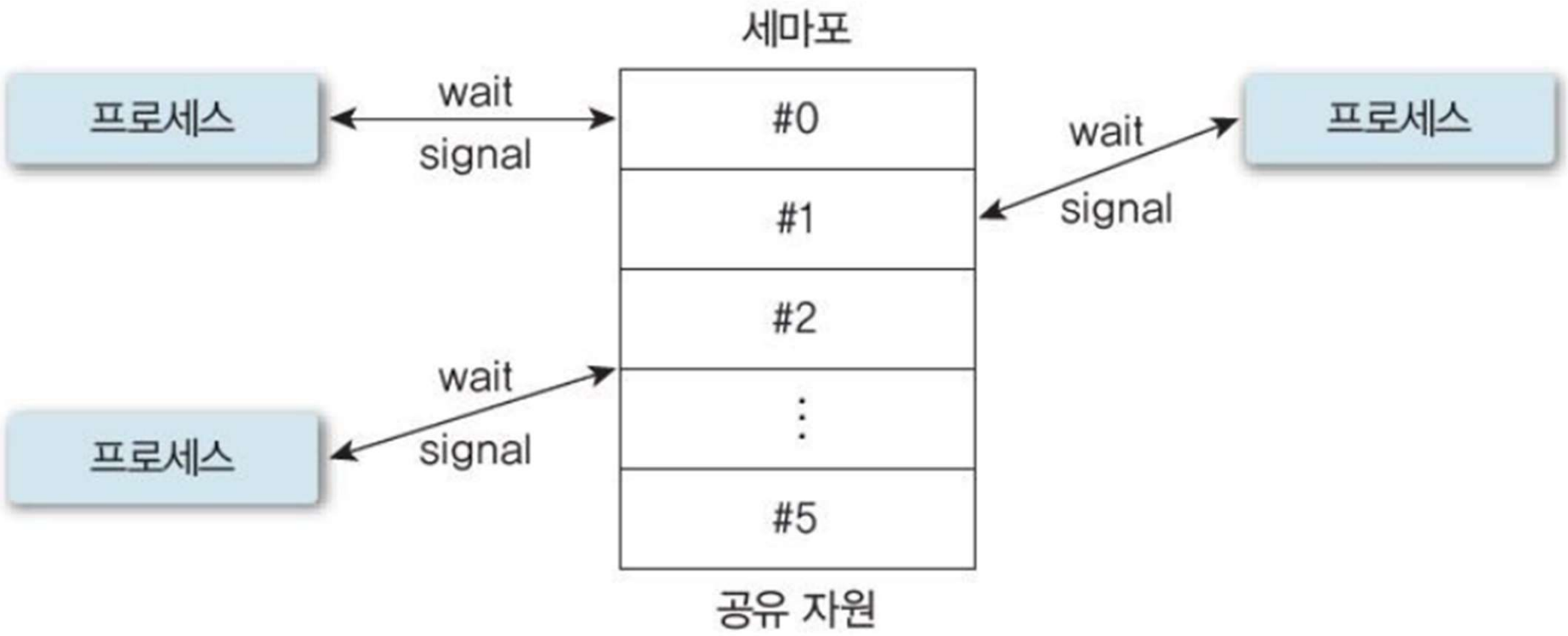
■ 세마포어 정의

```
P(S) : wait(S) {                    // S = 1로 초기화
          while S <= 0
               ;                    // 바쁜 대기, S > 0 때까지 대기
          S--;
     }

V(S) : signal(S) {
          S++;                      // 다른 프로세스의 접근 허용
     }
```



세마포어 공유자원 접근관리

- 세마포 공유 자원 접근 관리



세마포어 임계구역 문제 해결

- 세마포의 프로세스 n개의 임계 영역 문제 해결

```
do {
        wait(mutex);
                // 임계 영역
        signal(mutex);
                // 나머지 영역
} while (1);
```

- 세마포어 동기화

프로세스 P_1	프로세스 P_2
S_1;	wait(synch);
signal(synch);	S_2;

■ 세마포어의 구현

○ 세마포어 구조

```
struct semaphore {
        int count;
        queueType queue;
};
semaphore S;
```

▪ 세마포어 wait과 signal 연산

wait 연산

```
wait(S) {
    S -> count--;
    if (S -> count < 0) {
        add this process to S -> queue;      // 프로세스를 준비 큐에 추가
        block();                             // 프로세스 중단(일시정지)
    }
}
```

P와 V 연산은 세마포 S 수정하는 데 사용, 중단과 실행 동기화 구현. block 연산은 호출하는 프로세스를 중단, wakeup(P) 연산은 중단된 프로세스 P를 다 시 실행. wait 연산을 처음 수행하는 프로세스는 초깃값 S=1을 감소시켜 0이 됨. 그러 면 if 조건은 거짓이 되어 임계 영역을 수행. 이후 다른 프로세스가 계속 임계 영역에 진입하려고 P 연산 수행할 때마다 세마포 S 값 감소

signal 연산

```
signal(S) {
    S -> count++;
    if (S -> count <= 0) {
        remove a process P from S -> queue;  // 준비 큐에서 P 프로세스를 제거
        wakeup(P);                           // 신호를 보내 프로세스를 실행
    }
}
```

바쁜 대기 세마포에서는 절대 음수일 수 없지만, 여기에 구현된 세마포는 음수 값이 될 수 있다. 세마포 값이 음수이면 S 의 실제 값은 세마포에서 기다리는 프로세스 수를 나타내는데, 이는 wait 연산에서 감산과 검사 순서를 맞바꾸면서 나타남.

프로세스 준비 큐는 각 프로세스 제어 블록PCB의 링크 필드 정보 이용 쉽게 구현. 각 세마포는 정수 값과 PCB 등 열 포인터 포함. 준비 큐에서 프로세스 추가· 제거하는 가장 간단한 방법은 선입선출FIFO 큐 사용

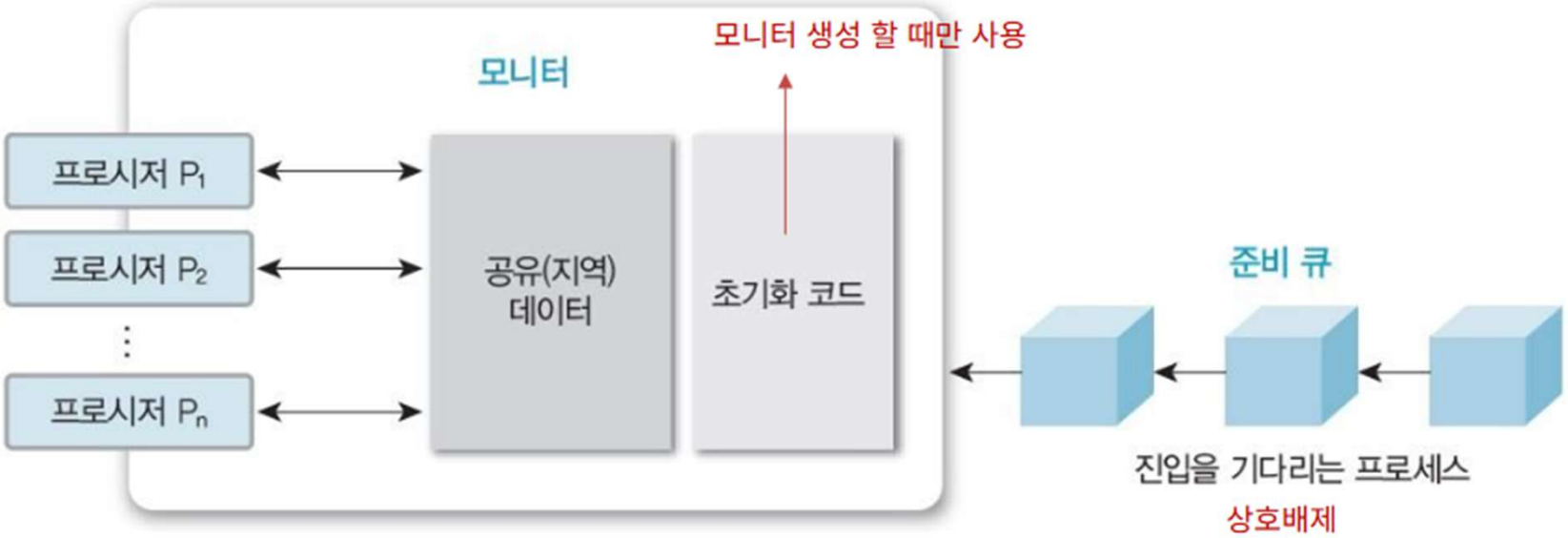
■ 두 프로세스가 동시에 동일한 세마포에서 wait와 signal 연산할 수 없도록 해야 하는 문제 해결 방법

- 단일 프로세서에서
  - wait와 signal 연산 수행 중 인터럽트 금지.
  - 인터럽트 금지하면 중간에 다른 프로세서의 명령어 실행이 끼어들지 않으므로 단일 프로세서 환경으로 활용 가능

- 다중 프로세서에서
  - 인터럽트 금지할 수 없어, 다른 프로세스의 명령어가 임의의 방법으로 끼어들 수 있음. 또 모든 프로세서에서 인터럽트 비활성화 곤란, 이는 심각한 성능 저하로 이어질 수 있음.
  - 하드웨어가 임계 영역 문제에 특별한 명령 제공 않으면 이를 소프트웨어적으로 해결해야 함, wait와 signal 연산으로는 바쁜 대기를 완전히 제거 할 수 없음.
  - 따라서 응용 프로그램의 진입 영역에서 임계 영역까지 바쁜 대기 제거 해야 함. 결국 바쁜 대기의 시기를 이동해야 하는 것. wait와 signal 연산이 임계 영역의 바쁜 대기 제한할 수 있으므로 임계 영역은 항상 비어 있고 바쁜 대기가 거의 일어나지 않는 것처럼 보임.
  - 바쁜 대기는 아주 짧은 기간 동안만 일어나지만, 어떤 응용 프로그램에서는 임계 영역 이 아주 길거나 항상 점유되어 있는 상황이 발생할 수 있음. 이때는 바쁜 대기가 매우 비효율적임

■ 모니터의 개념과 구조

- 세마포의 오용으로 여러 가지 오류가 쉽게 발생하면 프로그램 작성 곤란.
- 이런 단점 극복 위해 등장
- 핸슨Hansen 제안, 호Hoare 수정한 공유 자원과 이것의 임계 영역 관리 소프트웨어 구성체
- 사용자 사이에서 통신하려고 동기화하고, 자원에 배타적으로 접근할 수 있도록 프로세스가 사용하는 병행 프로그래밍 구조
- 모니터의 구조



모니터

- 모니터의 구조

```
Monitor monitor_name
{
                                        // 공유 데이터 변수 선언
        procedure P1(. . . .) {
                . . . .
        }
        procedure P2(. . . .) {
                . . . .
        }
        .
        .
        procedure Pn(. . . .) {
                . . . .
        }

        initialization_code(. . . .) {    // 코드 초기화
                . . . .
        }
}
```

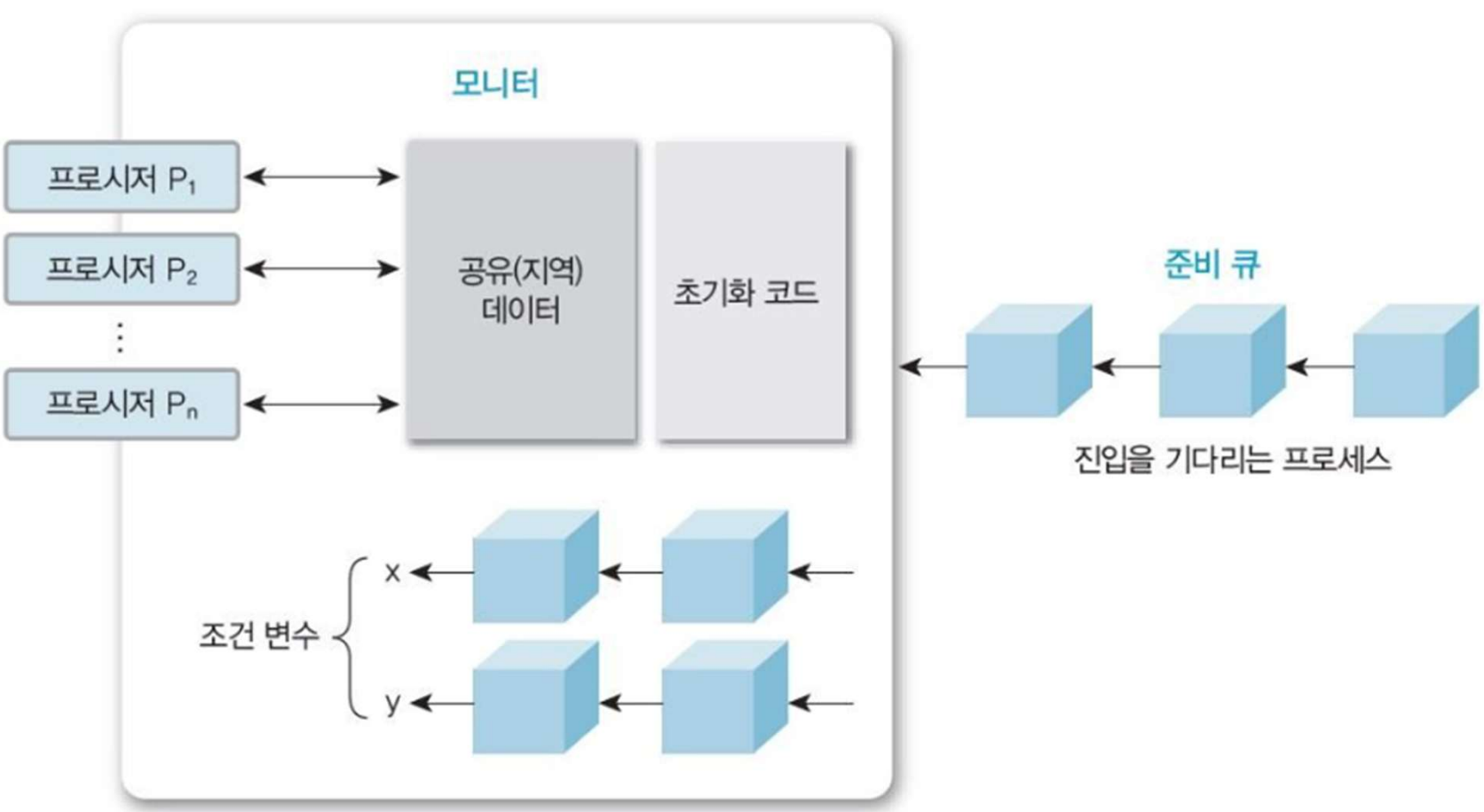
■ 조건 변수가 있는 모니터의 구조

- 모니터는 여러 측면에서 임계 영역과 비슷, 프로세스 실행 동안 상호배제와 동기화 제공
- 강력함 떨어져 동기화 방법 추가 정의해야 함
- 모니터 외부에 있는 프로세스가 모니터에 있는 프로세스 수행할 때까지 외부에서 기다려야 할 때는 특정 조건에 따라 실행 재개 결정
- 모니터는 조건 변수와 프로세스를 대기할 수 있는 상황 연관 시킴
- 하나 이상의 모니터 조건 변수 정의하여 모니터 안에서 작업 동기화

```
condition x, y;
```

모니터

- 조건 변수가 있는 모니터의 구조의 예



모니터와 세마포어의 비교

- **모니터**
- 모니터는 세마포어의 이런 사용 실수를 줄이기 위해 고안된 **고수준 동기화 구조**입니다. 공유 자원과 그것을 다루는 함수(프로시저)를 하나의 모듈로 묶고, 언어 또는 컴파일러 차원에서 한 번에 하나의 프로세스만 모니터 내부에 진입할 수 있도록 **자동으로 보장**합니다.
- 개발자는 그냥 모니터 내부에 함수를 작성하기만 하면 되고, 상호 배제를 위한 P·V 연산을 직접 관리할 필요가 없습니다. Java의 `synchronized` 키워드가 대표적인 모니터 구현입니다.
- 모니터는 내부적으로 **조건 변수(condition variable)** 를 사용합니다. 특정 조건이 만족될 때까지 대기할 때는 `wait()`, 조건이 충족되었음을 알릴 때는 `signal()`을 호출합니다. 이것이 세마포어의 P·V연산과 대응되는 개념이지만, 모니터 내부에서만 사용되므로 훨씬 안전합니다.

모니터와 세마포어의 비교

- **세마포어 (Semaphore)**
- 세마포어는 정수 변수 하나와 P·V 두 가지 연산으로 동기화를 구현하는 **저수준 도구**입니다. 개발자가 임계 구역의 앞뒤에 직접 P연산과 V연산을 배치해야 하며, 이 순서를 잘못 작성하면 교착 상태나 상호 배제 위반이 발생합니다. 즉, 동기화의 책임이 전적으로 **개발자**에게 있습니다.
- 예를 들어 P연산을 빠뜨리거나, V연산을 두 번 호출하거나, 순서를 바꿔 쓰는 실수가 쉽게 발생할 수 있고 이런 오류는 디버깅하기도 매우 어렵습니다.

- **핵심 차이: 누가 동기화를 책임지는가**
- 세마포어는 개발자가 올바른 위치에 P·V연산을 직접 삽입해야 하므로 실수의 여지가 큽니다. 반면 모니터는 구조 자체가 상호 배제를 내포하고 있어서 개발자는 로직에만 집중할 수 있습니다. 세마포어가 **자물쇠와 열쇠를 개발자가 직접 들고 다니는 방식**이라면, 모니터는 **자동문이 달린 방에 자원을 넣어두는 방식**이라고 볼 수 있습니다.